



MECHATRONIC SYSTEM INTEGRATION

MCTA 3203

WEEK 5 :

L298P MOTOR DRIVER SHIELD AND RELAY MODULE

SECTION 1

SEMESTER 2, 2025/2026

INSTRUCTOR:

**ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR. ZULKIFLI BIN ZAINAL
DR. WAHJU SEDIONO**

DATE OF SUBMISSION : 18 MAY 2026

GROUP 6

NAME	NO. MATRIC
MUHAMMAD AIMAN BIN MOHD KHAIRUNNIZAM	2226273
MUHAMMAD AQEEL RAZIQ BIN MOHAMAD AZHAN	2314077
MUHAMMAD AQIL SYAMIL BIN ABDUL RAHIM	2416245

Table of Contents

Abstract	3
Introduction	3
Materials and Equipment	4
Experimental setup	4
Methodology	5
Data Collection	6
Data Analysis	7
Results	8
Discussion	10
Conclusion	10
Recommendations	11
References	11
Appendices	12
Acknowledgments	17
Student's Declaration	18

Abstract

This experiment examines the operation, integration, and performance characteristics of solid-state and electromechanical power drivers used to control high-current mechatronic actuators. The primary objective was to regulate a 12V DC linear actuator using two different methods: an integrated L298P H-bridge motor driver shield and a 2-channel relay module. In the first phase, Pulse Width Modulation (PWM) was implemented via an Arduino Uno to control speed, while built-in logic inverters on the shield adjusted operational direction. In the second phase, a 2-channel relay module was used to achieve bidirectional position control. To ensure system safety, a limit switch and a push button were integrated into the relay circuit using internal pull-up configurations. This setup automatically stopped the actuator and turned on an LED indicator when specific extension or retraction points were reached. The experimental results confirmed that the L298P shield provides smooth speed control, while the relay circuit provides robust electrical isolation suitable for high-load automation applications.

Introduction

The integration of high-power actuators with low-voltage microcontrollers is a fundamental requirement in mechatronic system design. This laboratory experiment explores methods for controlling the speed, direction, and travel limits of a 12V DC linear actuator using two distinct power electronic interfaces: the L298P dual-channel H-bridge motor driver shield and an electromechanical 2-channel relay module. Microcontrollers like the Arduino Uno operate on low-voltage logic (5V) and cannot directly power high-current inductive loads, such as linear actuators, without risking overcurrent damage to the controller ports.

To address this, the L298P motor driver shield uses solid-state transistors configured as an H-bridge to safely route high-voltage motor power based on low-voltage control signals. It uses built-in inverters to simplify direction changes via single-pin logic and applies Pulse Width Modulation (PWM) to adjust the average voltage delivered to the load, thereby controlling motor speed. Conversely, the 2-channel relay module uses electromagnetic coils to provide complete galvanic isolation between the control logic and the high-power circuit. This method handles direction changes by altering wiring polarity through mechanical contacts. It is hypothesized that while the solid-state L298P driver will provide variable speed adjustments via PWM duty cycles,

the mechanical relay configuration will deliver full voltage and torque, using automated limit switches to prevent mechanical binding or motor stall at the stroke ends.

Materials and Equipment

1. 1x Arduino Uno Board + USB Interface Cable
2. 1x L298P Motor Driver Shield
3. 1x 2-Channel Relay Module
4. 1x 12V DC Linear Actuator
5. 1x Power Supply
6. 1x Limit Switch
7. 1x Push-Button Switch
8. 1x Breadboard & Jumper Wires
9. 1x LED
10. 1x 220 Ω Current-Limiting Resistor
11. 1x USB Interface Cable

Experimental setup

The experimental hardware configurations were assembled in two separate stages to test each power driver individually:

- **L298P Motor Driver Configuration:** The L298P driver shield was plugged directly onto the Arduino Uno headers, ensuring all pins aligned correctly. The 12V DC linear actuator was wired to the Motor A (MA) screw terminals. Crucially, the OPT jumper was removed from the shield to isolate the high-current motor power rail (VMS) from the Arduino logic circuitry, protecting the computer's USB port from back-EMF spikes. The variable DC power supply was set to 12V with a 3.0A current limit and connected to the VMS and GND terminals.
- **2-Channel Relay Configuration:** The L298P shield was removed, and a 2-channel relay module was wired to the Arduino Uno. The input control lines, IN1 and IN2, were connected to Digital Pins 7 and 8, respectively. The power supply's positive terminal (+12V) was connected to both Normally Open (NO) terminals, while the negative terminal (GND) was wired to both Normally Closed (NC) terminals. The two motor lines from the

linear actuator were connected to the corresponding Common (COM) terminals of the two relay channels.

- **Feedback and Sensor Integration:** For the automated relay control cycle, a limit switch was connected to Digital Pin 2, a push button was connected to Digital Pin 9, and an LED with a 220 Ω resistor was connected to Digital Pin 13. Both switches were configured using internal pull-up resistors (`INPUT_PULLUP`), connecting the signal pin to the switch's Normally Open terminal and the Common terminal to the Arduino GND.

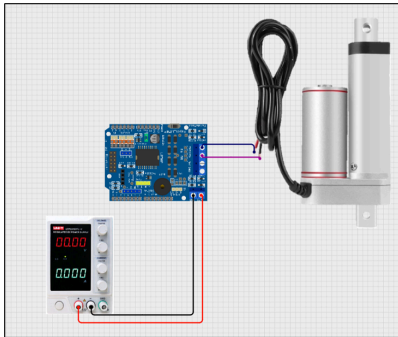


Figure shows setup for Task 1

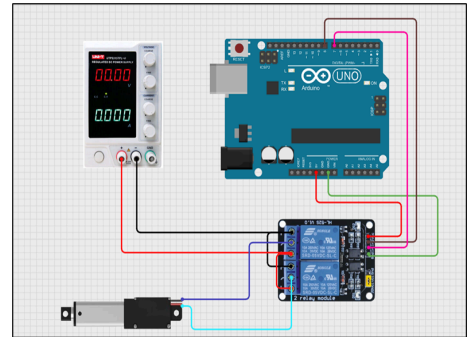


Figure shows setup for Task 2

Methodology

Part 1: DC Linear Actuator Control via L298P Shield

1. The L298P motor driver shield was mounted onto the Arduino Uno.
2. The actuator wires were connected to the Motor A output block, and the OPT jumper was removed to protect the system.
3. Control pins were configured in the Arduino IDE, mapping the speed pin (ENA) to Digital Pin 6 and the direction pin (INA) to Digital Pin 7.
4. An Arduino script was uploaded to set direction pin INA to `LOW` and write a PWM value of 150 to pin ENA using `analogWrite()`. The actuator's physical movement was observed for 3000 ms.
5. The logic state of pin INA was then flipped to `HIGH` while maintaining the PWM speed at 150. The resulting directional change was recorded over another 3000 ms interval.

Part 2: Automated Actuator Cycle via 2-Channel Relay Module

1. The circuit was reconfigured using the 2-channel relay module, linking relay inputs IN1 and IN2 to Arduino Pins 7 and 8.

2. Cross-wire bridges were added between the NC and NO terminals across both channels to construct an electromechanical H-bridge capable of reversing voltage polarity.
3. The limit switch (extension boundary) and push-button switch (retraction boundary) were integrated into the setup using internal pull-up logic (`INPUT_PULLUP`).
4. An automation script was executed. The program initiated the extension cycle by setting Pin 7 `HIGH` and Pin 8 `LOW`, running a continuous `while()` loop to monitor the limit switch.
5. Once the actuator struck the limit switch, the signal pin dropped to `LOW`, breaking the loop. The script immediately cut power to both relays, turned on the Pin 13 LED, and paused the system for 2000 ms.
6. The retraction cycle was then initiated by setting Pin 7 `LOW` and Pin 8 `HIGH`. The script monitored the push button until a `LOW` logic state was detected, at which point it stopped the actuator, illuminated the LED, and reset the loop.

Data Collection

- L298P Driver Test Constants:
 - Fixed Power Supply Output: 12.00V DC
 - Power Supply Current Limit Selection: 3.00A
 - Assigned Fixed Speed Parameter: Value 150 out of 255 (equivalent to a ~58.8% Duty Cycle)
- Observed Actuator Movement Cycles (L298P Driver Shield):
 - INA state = `LOW` → Actuator Extended smoothly at a moderated intermediate velocity.
 - INA state = `HIGH` → Actuator Retracted smoothly at the same intermediate velocity.
- Switch Logic Operational Parameters (Relay Automation Mode):
 - Open / Idle State Switch Reading: `HIGH` (5V internal pull-up profile)
 - Closed / Triggered State Switch Reading: `LOW` (0V direct path to GND)
- Serial Terminal Diagnostic Logs (Relay Automation Mode):

```
Linear Actuator + Limit Switch + Button System Ready
Extending...
Extend limit reached! LED ON, actuator stopped.
Retracting...
Retract limit reached! LED ON, actuator stopped.
```

Data Analysis

- **Pulse Width Modulation (PWM) Calculations:** The L298P shield controls the speed of the linear actuator by adjusting the PWM duty cycle on the enable pin (ENA). The microcontroller outputs a high-frequency square wave that cycles between 5V (ON) and 0V (OFF). The duty cycle represents the percentage of time the signal remains high during each period:

$$\text{Duty Cycle} = \left(\frac{150}{255} \right) \times 100\% = 58.82\%$$

Given an assigned experimental value of 150:

$$\text{Duty Cycle (\%)} = \left(\frac{\text{PWM Value}}{255} \right) \times 100\%$$

This duty cycle determines the average voltage (V_{avg}) delivered to the linear actuator from the 12V external power supply:

$$\begin{aligned} V_{\text{avg}} &= \text{Duty Cycle} \times V_{\text{source}} \\ V_{\text{avg}} &= 0.5882 \times 12.0\text{V} = 7.06\text{V} \end{aligned}$$

Operating at an average of 7.06V allowed the actuator to travel at a moderated speed, demonstrating the L298P's suitability for applications requiring variable speed control.

- **Relay H-Bridge Polarity Analysis:**

The 2-channel relay module was configured to act as an electromechanical H-bridge. Because the positive supply line was wired to the NO terminals and the negative line to the NC terminals, changing the digital logic states of the relays reversed the electrical polarity across the actuator:

Forward Drive (Extend): Relay 1 (HIGH) $\rightarrow$ COM1 = NO(+12V); Relay 2 (LOW) $\rightarrow$ COM2 = NC(GND)

Reverse Drive (Retract): Relay 1 (LOW) $\rightarrow$ COM1 = NC(GND); Relay 2 (HIGH) $\rightarrow$ COM2 = NO(+12V)

Active Stop (Safe State): Relay 1 (LOW) $\rightarrow$ COM1 = NC(GND); Relay 2 (LOW) $\rightarrow$ COM2 = NC(GND)

Unlike the L298P solid-state driver, this relay configuration delivers the full 12.00V supply to the motor, ensuring maximum mechanical torque during travel.

Results

The functional performance of both control methods is summarized in Table 1 and Table 2 below.

Direction Pin (INA)	Speed Pin (ENA)	Calculated Duty Cycle	Actuator Physical Behavior
LOW	Mapped to Pin 6 (analogWrite: 150)	58.82%	Extends outward at medium speed
HIGH	Mapped to Pin 6 (analogWrite: 150)	58.82%	Retracts inward at medium speed
LOW or HIGH	analogWrite: 0	0.00%	Stops moving immediately

Table 1

Relay 1 (Pin 7)	Relay 2 (Pin 8)	Limit Switch (Pin 2)	Button Switch (Pin 9)	Status LED (Pin 13)	Actuator Physical State
HIGH	LOW	HIGH (Open)	HIGH (Open)	LOW (OFF)	Extends under full 12V power
LOW	LOW	LOW (Triggered)	HIGH (Open)	HIGH (ON)	Stops instantly at extension limit
LOW	HIGH	HIGH (Open)	HIGH (Open)	LOW (OFF)	Retracts under full 12V power
LOW	LOW	HIGH (Open)	LOW (Pressed)	HIGH (ON)	Stops instantly at retraction limit

Table 2

Discussion

- **Analysis of Performance Characteristics:** Both methods successfully controlled the 12V linear actuator, but they suit different mechatronic requirements. The L298P shield provides smooth, variable speed control through PWM duty cycles, making it ideal for systems that require precise velocity adjustments. However, it introduces a small voltage drop across its internal transistors. In contrast, the relay module delivers full supply voltage and maximum torque, operating with high electrical efficiency and complete galvanic isolation. This makes it highly effective for heavy-duty applications where variable speed is not required.
- **Switch Logic and Debouncing Behavior:** Using the internal `INPUT_PULLUP` resistors simplified the circuit design by eliminating the need for external hardware resistors. The `while()` loop structure locked the execution state effectively until the switches were triggered. However, mechanical switches can experience contact bounce, which can cause microsecond-level signal oscillations. In high-precision tracking loops, software debouncing filters or small hardware capacitors should be added to prevent false boundary triggers.
- **Safety Rules and Structural Adjustments:** Removing the OPT jumper was an essential safety step. Without this separation, high-current inductive surges (back-EMF) from the linear actuator could flow back into the Arduino logic lines, potentially resetting the microcontroller or damaging the connected computer's USB port.

Conclusion

The experiment successfully demonstrated how to control a 12V DC linear actuator using both solid-state and electromechanical power drivers. The L298P shield proved effective for speed and direction adjustments via PWM control and simplified single-pin directional logic. Meanwhile, the 2-channel relay module provided a robust method for high-torque bidirectional control, and the integration of boundary switches ensured safe operation by automatically preventing over-travel. Ultimately, all core objectives were met, providing practical insight into power electronic interfaces for mechatronic applications.

Recommendations

- **Inductive Surge Suppression:** Add flyback diodes across the relay contacts or motor terminals to suppress inductive voltage spikes, which helps prolong the operational lifespan of the mechanical relay contacts.
- **Enhanced Safety Redundancy:** Implement hardwired, normally closed emergency stop switches directly along the main power line to cut actuator power immediately if a software loop freezes.
- **Current Monitoring Integration:** Connect a low-resistance shunt resistor or a Hall-effect current sensor to the execution path. This allows the system to detect structural obstructions through overcurrent spikes and automatically stop the actuator to protect the motor from burning out.

References

1. Cytron Technologies: *L298P Dual Channel Motor Driver Shield Technical Guide*, 2023.
2. Week 05-2026 - L298P motor driver and Relay.docx

Appendices

Appendix A: Final Task 1 Motor Shield Speed Control Code

```
const int ENA = 6; // Control speed of motor A
const int INA = 7; // Control direction of motor A

void setup()
{
  pinMode(ENA, OUTPUT);
  pinMode(INA, OUTPUT);
}

void loop()
{
  // Drive Forward at Medium Speed
  digitalWrite(INA, LOW);
  analogWrite(ENA, 150);
  delay(3000);

  // Drive Backward at Medium Speed
  digitalWrite(INA, HIGH);
  analogWrite(ENA, 150);
  delay(3000);
}
```

Appendix B: Final Task 2 Automated Relay Control Code

```
/*
 * =====
 * Linear Actuator with Limit Switch + Button Switch & LED
 * =====
 */

const int relay1 = 7;           // Relay IN1 - controls direction
const int relay2 = 8;           // Relay IN2 - controls direction
const int limitSwitch = 2;      // Limit switch - detects fully extended
const int buttonSwitch = 9;     // Button - detects fully retracted
const int ledPin = 13;          // LED indicator

void setup()
{
```

```

pinMode(relay1, OUTPUT);
pinMode(relay2, OUTPUT);
pinMode(ledPin, OUTPUT);

// Both switches use internal pull-up resistors
pinMode(limitSwitch, INPUT_PULLUP);
pinMode(buttonSwitch, INPUT_PULLUP);

// Start with everything off
stopActuator();
digitalWrite(ledPin, LOW);

Serial.begin(9600);
Serial.println("Linear Actuator + Limit Switch + Button System Ready");
}

void loop()
{
    // --- EXTEND until limit switch is hit ---
    Serial.println("Extending...");
    digitalWrite(ledPin, LOW);
    extendActuator();

    while (digitalRead(limitSwitch) == HIGH) {
        delay(10); // Keep extending
    }

    // Limit switch triggered!
    stopActuator();
    digitalWrite(ledPin, HIGH);
    Serial.println("Extend limit reached! LED ON, actuator stopped.");
    delay(2000);

    // --- RETRACT until button is pressed ---
    Serial.println("Retracting...");
    digitalWrite(ledPin, LOW);
    retractActuator();

    while (digitalRead(buttonSwitch) == HIGH) {
        delay(10); // Keep retracting
    }

    // Button pressed!
    stopActuator();

```

```
    digitalWrite(ledPin, HIGH);
    Serial.println("Retract limit reached! LED ON, actuator stopped.");
    delay(2000);
}

// --- Actuator Control Functions ---

void extendActuator()
{
    digitalWrite(relay1, HIGH);
    digitalWrite(relay2, LOW);
}

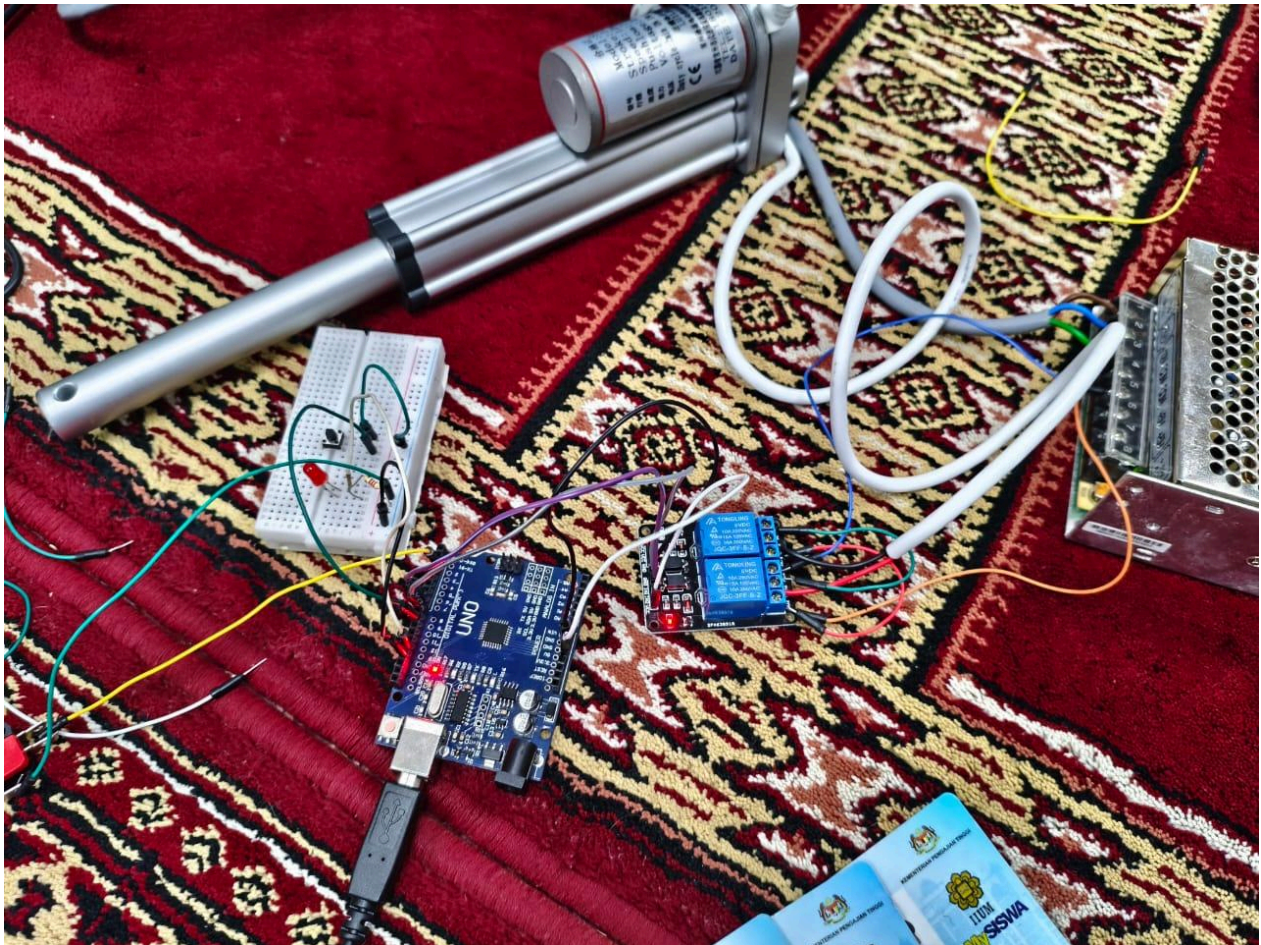
void retractActuator()
{
    digitalWrite(relay1, LOW);
    digitalWrite(relay2, HIGH);
}

void stopActuator()
{
    digitalWrite(relay1, LOW);
    digitalWrite(relay2, LOW);
}
```

Appendix C: Picture of completed circuit connection for Task 1



Appendix D: Picture of completed circuit connection for Task 2



Acknowledgments

All praise belongs to Allah SWT for granting us the strength and wisdom to complete the "L298P Motor Driver Shield and relay module " experiment. We are grateful for His guidance in navigating the challenges faced throughout the circuit integration process. We also wish to acknowledge the vital support of our lecturers and fellow peers. Their academic supervision and practical insights into mechatronic system integration were essential to the successful operation of our project and the completion of this report.

Student's Declaration

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.

We hereby certify that this report has not been done by only one individual and all of us have contributed to the report.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Signature: 

Read [/]

Name: Muhammad Aqeel Raziq Bin Mohamad Azhan

Understand [/]

Matric Number: 2314077

Agree [/]

Signature: 

Read [/]

Name: Muhammmad Aiman Bin Mohd Khairunnizam

Understand [/]

Matric Number: 2226273

Agree [/]

Signature: 

Read [/]

Name: Muhammad Aqil Syamil bin Abdul Rahim

Understand [/]

Matric Number: 2416245

Agree [/]